

```

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const mongoose = require('mongoose');
const { OAuth2Client } = require('google-auth-library');

const app = express();

app.set('trust proxy', 1);

app.use(helmet({
  contentSecurityPolicy: false,
  crossOriginEmbedderPolicy: false,
}));

const allowedOrigins = [
  'https://aporialab.space',
  'https://www.aporialab.space',
  'https://aporialab-frontend.vercel.app',
  'http://localhost:5173',
  'http://localhost:3000',
];

const corsOptions = {
  origin: (origin, callback) => {
    if (!origin) return callback(null, true);
    if (origin.endsWith('.vercel.app')) return callback(null, true);
    if (allowedOrigins.indexOf(origin) !== -1) return callback(null, true);
    return callback(new Error('Not allowed by CORS'));
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization', 'Accept'],
  credentials: false
};

app.use(cors(corsOptions));
app.options('*', cors(corsOptions));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

const generalLimiter = rateLimit({

```

```

    windowMs: 15 * 60 * 1000,
    max: 100,
    message: { success: false, message: 'ثير من الطلبات. حاول مرة أخرى بعد 15 دقيقة',
    standardHeaders: true,
    legacyHeaders: false,
  });

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { success: false, message: 'من محاولات تسجيل الدخول. حاول بعد 15 دقيقة',
  standardHeaders: true,
  legacyHeaders: false,
  skipSuccessfulRequests: true,
});

const googleAuthLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: { success: false, message: 'من محاولات تسجيل الدخول. حاول بعد 15 دقيقة',
  standardHeaders: true,
  legacyHeaders: false,
  skipSuccessfulRequests: true,
});

app.use('/api/', generalLimiter);

const JWT_SECRET = process.env.JWT_SECRET || 'aporialab-secret-key-2026';
const MONGODB_URI = process.env.MONGODB_URI;
const GOOGLE_CLIENT_ID = process.env.GOOGLE_CLIENT_ID || '';

const googleClient = new OAuth2Client(GOOGLE_CLIENT_ID);

const VALID_DURATIONS = {
  '12h': 12 * 60 * 60 * 1000,
  '24h': 24 * 60 * 60 * 1000,
  '3d': 3 * 24 * 60 * 60 * 1000,
  '7d': 7 * 24 * 60 * 60 * 1000,
};

const MAX_REACTIONS_PER_USER = 2;

const REPUTATION_REWARDS = {
  CREATE_DISCUSSION: 10,
  CREATE_COMMENT: 2,
  RECEIVE_UPVOTE: 5,
  RECEIVE_LOGICAL: 3,
};

```

```

    RECEIVE_INSPILING: 2,
    RECEIVE_ILLOGICAL: -2,
    RECEIVE_UNCLEAR: -1,
  };

let cachedConnection = null;
async function connectDB() {
  if (cachedConnection && mongoose.connection.readyState === 1) return cachedConn
  if (!MONGODB_URI) throw new Error('MONGODB_URI not configured');
  try {
    cachedConnection = await mongoose.connect(MONGODB_URI, {
      bufferCommands: false,
      serverSelectionTimeoutMS: 10000,
    });
    return cachedConnection;
  } catch (error) {
    console.error('MongoDB connection error:', error.message);
    cachedConnection = null;
    throw error;
  }
}

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true, maxlength: 100 },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, default: '' },
  googleId: { type: String, default: null, sparse: true, index: true },
  authProvider: { type: String, enum: ['local', 'google'], default: 'local' },
  emailVerified: { type: Boolean, default: false },
  avatar: { type: String, default: '' },
  bio: { type: String, default: '', maxlength: 500 },
  reputation: { type: Number, default: 0 },
  role: { type: String, enum: ['user', 'moderator', 'admin'], default: 'user' },
  isFoundingMember: { type: Boolean, default: false },
}, { timestamps: true });

const EditHistoryEntrySchema = new mongoose.Schema({
  editedAt: { type: Date, default: Date.now },
  editedBy: {
    _id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: String,
  },
  previousTitle: String,
  previousContent: String,
  reason: { type: String, default: '' },
}, { _id: false });

```

```

const DiscussionSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxlength: 200 },
  content: { type: String, required: true, maxlength: 10000 },
  category: { type: String, enum: ['beginner', 'intermediate', 'advanced'], default: 'beginner' },
  tags: [{ type: String, maxlength: 50 }],
  author: {
    _id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: String, avatar: String, reputation: Number, isFoundingMember: Boolean
  },
  views: { type: Number, default: 0 },
  upvotes: [{ type: String }],
  commentCount: { type: Number, default: 0 },
  expiresAt: { type: Date, default: null },
  duration: { type: String, enum: ['12h', '24h', '3d', '7d', null], default: null },
  stanceStats: {
    pro: { type: Number, default: 0 },
    con: { type: Number, default: 0 },
    neutral: { type: Number, default: 0 },
  },
  editHistory: [EditHistoryEntrySchema],
  editedAt: { type: Date, default: null },
  editsCount: { type: Number, default: 0 },
}, { timestamps: true });

```

```

const CommentSchema = new mongoose.Schema({
  discussionId: { type: mongoose.Schema.Types.ObjectId, ref: 'Discussion', required: true },
  content: { type: String, required: true, maxlength: 5000 },
  author: {
    _id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: String, avatar: String, reputation: Number, isFoundingMember: Boolean
  },
  stance: { type: String, enum: ['pro', 'con', 'neutral'], required: true },
  upvotes: [{ type: String }],
  reactions: {
    logical: [{ type: String }],
    illogical: [{ type: String }],
    inspiring: [{ type: String }],
    unclear: [{ type: String }],
  },
  qualityScore: { type: Number, default: 0 },
  parentCommentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Comment', default: null },
  isReply: { type: Boolean, default: false },
  editedAt: { type: Date, default: null },
}, { timestamps: true });

```

```

const CircleSchema = new mongoose.Schema({
  name: { type: String, required: true, maxlength: 100 },

```

```

description: { type: String, maxlength: 500 },
category: String,
members: { type: Number, default: 0 },
isPrivate: { type: Boolean, default: false },
icon: String,
color: String,
bannerColor: String,
tags: [{ type: String }],
memberIds: [{ type: String }],
pendingRequests: [{
  userId: String,
  userName: String,
  userAvatar: String,
  requestedAt: { type: Date, default: Date.now },
  message: String,
}],
discussionCount: { type: Number, default: 0 },
createdBy: {
  _id: String,
  name: String,
},
}, { timestamps: true });

const NotificationSchema = new mongoose.Schema({
  recipient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true,
  sender: {
    _id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: String,
    avatar: String,
  },
  type: {
    type: String,
    enum: [
      'comment',
      'reply',
      'discussion_upvote',
      'comment_upvote',
      'reaction_logical',
      'reaction_inspiring',
      'circle_join_request',
      'circle_approved',
      'circle_rejected',
    ],
    required: true,
  },
  title: { type: String, required: true },
  message: { type: String, required: true },

```

```

    link: { type: String, required: true },
    metadata: {
      discussionId: { type: mongoose.Schema.Types.ObjectId, ref: 'Discussion' },
      commentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Comment' },
      circleId: { type: mongoose.Schema.Types.ObjectId, ref: 'Circle' },
    },
    isRead: { type: Boolean, default: false, index: true },
  }, { timestamps: true });

NotificationSchema.index({ createdAt: 1 }, { expireAfterSeconds: 60 * 24 * 60 * 6

const User = mongoose.models.User || mongoose.model('User', UserSchema);
const Discussion = mongoose.models.Discussion || mongoose.model('Discussion', Dis
const Comment = mongoose.models.Comment || mongoose.model('Comment', CommentSchem
const Circle = mongoose.models.Circle || mongoose.model('Circle', CircleSchema);
const Notification = mongoose.models.Notification || mongoose.model('Notification

// =====
// 🛎 NOTIFICATIONS HELPER
// =====
async function createNotification({ recipient, sender, type, title, message, link
  try {
    if (sender && sender._id && recipient.toString() === sender._id.toString()) {
      return null;
    }
    const notification = await Notification.create({
      recipient,
      sender: sender ? {
        _id: sender._id,
        name: sender.name,
        avatar: sender.avatar,
      } : undefined,
      type,
      title,
      message,
      link,
      metadata,
    });
    return notification;
  } catch (err) {
    console.error('Create notification error:', err);
    return null;
  }
}

function sanitizeString(str, maxLen) {
  if (typeof str !== 'string') return '';

```

```

    maxLen = maxLen || 1000;
    return str.trim().slice(0, maxLen).replace(/[\x00-\x1F\x7F]/g, '');
}

function isValidEmail(email) {
    return typeof email === 'string' && /^[^\\s@]+@[^\\s@]+\\.([\\s@]+$/\\.test(email);
}

function isDiscussionExpired(discussion) {
    if (!discussion.expiresAt) return false;
    return new Date() > new Date(discussion.expiresAt);
}

function calculateQualityScore(comment) {
    const upvotes = (comment.upvotes || []).length;
    const logical = (comment.reactions?.logical || []).length;
    const illogical = (comment.reactions?.illogical || []).length;
    const inspiring = (comment.reactions?.inspiring || []).length;
    const unclear = (comment.reactions?.unclear || []).length;

    const positive = upvotes * 3 + logical * 2 + inspiring * 1.5;
    const negative = illogical * 2 + unclear * 0.5;

    return Math.round(positive - negative);
}

function userToResponse(user) {
    return {
        id: user._id.toString(),
        _id: user._id.toString(),
        name: user.name,
        email: user.email,
        avatar: user.avatar,
        bio: user.bio,
        reputation: user.reputation,
        role: user.role,
        isFoundingMember: user.isFoundingMember,
        authProvider: user.authProvider,
        emailVerified: user.emailVerified
    };
}

async function updateAuthorReputation(authorId, points) {
    if (!authorId || !points) return;
    try {
        await User.findByIdAndUpdate(authorId, { $inc: { reputation: points } });
    } catch (e) {

```

```

        console.error('Reputation update failed:', e.message);
    }
}

app.use(async (req, res, next) => {
    try { await connectDB(); next(); } catch (e) { res.status(503).json({ success:
});});

function authMiddleware(req, res, next) {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) return res.status(401).js
    try {
        req.user = jwt.verify(authHeader.split(' ')[1], JWT_SECRET);
        next();
    } catch (e) {
        return res.status(401).json({ success: false, message: 'جى تسجيل الدخول مجدداً'
    }
}

app.get('/', (req, res) => res.json({
    name: 'AporiaLab API',
    version: '4.1.0',
    status: 'running',
    database: 'MongoDB',
    security: 'enhanced',
    auth: 'local + google',
    features: 'stances + reactions + timer + replies + edit history + reputation sy
}));

app.get('/api/health', async (req, res) => {
    try {
        const userCount = await User.countDocuments();
        const discussionCount = await Discussion.countDocuments();
        res.json({ status: 'OK', timestamp: new Date().toISOString(), database: 'conn
    } catch (error) {
        res.status(500).json({ status: 'ERROR', message: error.message });
    }
});

app.get('/api/stats', async (req, res) => {
    try {
        const userCount = await User.countDocuments();
        const discussionCount = await Discussion.countDocuments();
        const circleCount = await Circle.countDocuments();
        const commentCount = await Comment.countDocuments();
        res.json({
            success: true,

```



```

    stats: {
      users: userCount,
      discussions: discussionCount,
      circles: circleCount,
      comments: commentCount,
      contributions: discussionCount + commentCount
    }
  });
} catch (error) {
  console.error('Stats error:', error);
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.post('/api/auth/google', googleAuthLimiter, async (req, res) => {
  try {
    if (!GOOGLE_CLIENT_ID) {
      return res.status(503).json({ success: false, message: 'الخادم Google خدمة' });
    }

    const credential = req.body.credential;
    if (!credential || typeof credential !== 'string') {
      return res.status(400).json({ success: false, message: 'صيحة Google بيانات' });
    }

    let payload;
    try {
      const ticket = await googleClient.verifyIdToken({
        idToken: credential,
        audience: GOOGLE_CLIENT_ID,
      });
      payload = ticket.getPayload();
    } catch (verifyError) {
      console.error('Google token verification failed:', verifyError.message);
      return res.status(401).json({ success: false, message: 'Google فشل التحقق من' });
    }

    if (!payload || !payload.email) {
      return res.status(401).json({ success: false, message: 'كتملة Google بيانات' });
    }

    if (!payload.email_verified) {
      return res.status(403).json({ success: false, message: 'بريدك الإلكتروني في' });
    }

    const googleId = payload.sub;
    const email = payload.email.toLowerCase();
  }
});

```

```

const googleName = payload.name || payload.given_name || email.split('@')[0];
const googlePicture = payload.picture || '';

let user = await User.findOne({ googleId });

if (!user) {
  user = await User.findOne({ email });

  if (user) {
    user.googleId = googleId;
    user.authProvider = 'google';
    user.emailVerified = true;
    if (!user.avatar && googlePicture) user.avatar = googlePicture;
    await user.save();
  } else {
    const randomPassword = await bcrypt.hash(googleId + Date.now().toString());
    user = await User.create({
      name: sanitizeString(googleName, 100),
      email,
      password: randomPassword,
      googleId,
      authProvider: 'google',
      emailVerified: true,
      avatar: googlePicture || ('https://api.dicebear.com/7.x/avataaars/svg?s'
      bio: '',
      reputation: 0,
      role: 'user',
      isFoundingMember: false
    });
  }
}

const token = jwt.sign(
  { userId: user._id.toString(), email: user.email },
  JWT_SECRET,
  { expiresIn: '30d' }
);

res.json({ success: true, token, user: userToResponse(user) });
} catch (error) {
  console.error('Google auth error:', error);
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.post('/api/auth/login', authLimiter, async (req, res) => {
  try {

```

```

const email = sanitizeString(req.body.email, 200).toLowerCase();
const password = req.body.password;
if (!isValidEmail(email) || !password) return res.status(400).json({ success:
const user = await User.findOne({ email });
if (!user) return res.status(401).json({ success: false, message: 'غير صحيحة'
if (user.authProvider === 'google' && !user.password) {
  return res.status(401).json({ success: false, message: 'حساب يُسجّل الدخول بـ'
}
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) return res.status(401).json({ success: false, message: 'ر صحيحة'
const token = jwt.sign({ userId: user._id.toString(), email: user.email }, JW
res.json({ success: true, token, user: userToResponse(user) });
} catch (error) {
  console.error('Login error:', error);
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

```

```

app.post('/api/auth/register', authLimiter, async (req, res) => {
  try {
    const name = sanitizeString(req.body.name, 100);
    const email = sanitizeString(req.body.email, 200).toLowerCase();
    const password = req.body.password;
    if (!name || name.length < 2) return res.status(400).json({ success: false, m
    if (!isValidEmail(email)) return res.status(400).json({ success: false, messa
    if (!password || password.length < 6) return res.status(400).json({ success:
    if (password.length > 200) return res.status(400).json({ success: false, mess
    const existing = await User.findOne({ email });
    if (existing) return res.status(400).json({ success: false, message: 'ل مسبقاً'
    const hashedPassword = await bcrypt.hash(password, 10);
    const newUser = await User.create({
      name, email, password: hashedPassword,
      authProvider: 'local',
      emailVerified: false,
      avatar: 'https://api.dicebear.com/7.x/avataaars/svg?seed=' + encodeURICompo
      bio: '', reputation: 0, role: 'user', isFoundingMember: false
    });
    const token = jwt.sign({ userId: newUser._id.toString(), email: newUser.email
    res.status(201).json({ success: true, token, user: userToResponse(newUser) })
  } catch (error) {
    console.error('Register error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

```

```

app.get('/api/auth/me', authMiddleware, async (req, res) => {
  try {

```

```

    const user = await User.findById(req.user.userId);
    if (!user) return res.status(404).json({ success: false, message: 'غير موجود' });
    res.json({ success: true, user: userToResponse(user) });
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.get('/api/discussions', async (req, res) => {
  try {
    const sort = req.query.sort || 'trending';
    const level = req.query.level;
    const filter = req.query.filter;
    const page = Math.max(1, parseInt(req.query.page) || 1);
    const limit = Math.min(50, Math.max(1, parseInt(req.query.limit) || 10));
    const query = {};
    if (level && level !== 'all') query.category = level;
    if (filter && filter !== 'all') query.category = filter;
    let sortObj = { createdAt: -1 };
    if (sort === 'trending') sortObj = { views: -1, createdAt: -1 };
    else if (sort === 'featured') sortObj = { upvotes: -1 };
    else if (sort === 'live') sortObj = { commentCount: -1 };
    const total = await Discussion.countDocuments(query);
    const discussions = await Discussion.find(query).sort(sortObj).skip((page - 1) * limit).limit(limit);
    res.json({
      success: true,
      discussions: discussions.map(d => Object.assign({}, d, {
        _id: d._id.toString(),
        isExpired: isDiscussionExpired(d)
      })),
      pagination: { page, pages: Math.ceil(total / limit), total }
    });
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.get('/api/discussions/:id', async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).json({ success: false, message: 'ID غير صالح' });
    const discussion = await Discussion.findByIdAndUpdate(req.params.id, { $inc: { views: 1 } });
    if (!discussion) return res.status(404).json({ success: false, message: 'موجود' });

    const comments = await Comment.find({ discussionId: req.params.id })
      .sort({ qualityScore: -1, createdAt: -1 })
      .lean();
  }
});

```

```

const enrichedComments = comments.map(c => ({
  ...c,
  _id: c._id.toString(),
  parentCommentId: c.parentCommentId ? c.parentCommentId.toString() : null,
  reactions: {
    logical: c.reactions?.logical || [],
    illogical: c.reactions?.illogical || [],
    inspiring: c.reactions?.inspiring || [],
    unclear: c.reactions?.unclear || [],
  }
}));

res.json({
  success: true,
  discussion: Object.assign({}, discussion, {
    _id: discussion._id.toString(),
    comments: enrichedComments,
    isExpired: isDiscussionExpired(discussion)
  })
});
} catch (error) {
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.get('/api/discussions/:id/history', async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
    }
    const discussion = await Discussion.findById(req.params.id).lean();
    if (!discussion) return res.status(404).json({ success: false, message: 'وجود' });

    res.json({
      success: true,
      editHistory: discussion.editHistory || [],
      editsCount: discussion.editsCount || 0,
      editedAt: discussion.editedAt,
      currentTitle: discussion.title,
      currentContent: discussion.content,
    });
  } catch (error) {
    console.error('Get history error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

```

```

app.post('/api/discussions', authMiddleware, async (req, res) => {
  try {
    const title = sanitizeString(req.body.title, 200);
    const content = sanitizeString(req.body.content || req.body.description, 1000)
    const level = ['beginner', 'intermediate', 'advanced'].includes(req.body.level)
    const tags = Array.isArray(req.body.tags) ? req.body.tags.slice(0, 10).map(t => t.toLowerCase()) : []
    const duration = req.body.duration && Object.keys(VALID_DURATIONS).includes(req.body.duration) ? req.body.duration : null

    if (!title || title.length < 5) return res.status(400).json({ success: false, message: 'عنوان غير صحيح' })
    if (!content || content.length < 10) return res.status(400).json({ success: false, message: 'محتوى غير صحيح' })

    const user = await User.findById(req.user.userId);
    if (!user) return res.status(404).json({ success: false, message: 'غير موجود' })

    const expiresAt = duration ? new Date(Date.now() + VALID_DURATIONS[duration]) : null

    const newDiscussion = await Discussion.create({
      title, content, category: level, tags,
      author: {
        _id: user._id,
        name: user.name,
        avatar: user.avatar,
        reputation: user.reputation,
        isFoundingMember: user.isFoundingMember
      },
      duration,
      expiresAt,
      stanceStats: { pro: 0, con: 0, neutral: 0 },
      editHistory: [],
      editsCount: 0,
    });

    await updateAuthorReputation(user._id, REPUTATION_REWARDS.CREATE_DISCUSSION);
    res.status(201).json({
      success: true,
      discussion: Object.assign({}, newDiscussion.toObject(), {
        _id: newDiscussion._id.toString(),
        isExpired: false
      })
    });
  } catch (error) {
    console.error('Create discussion error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.patch('/api/discussions/:id', authMiddleware, async (req, res) => {
  try {

```

```

if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
  return res.status(400).json({ success: false, message: 'معرفة غير صحيح' });
}
const discussion = await Discussion.findById(req.params.id);
if (!discussion) return res.status(404).json({ success: false, message: 'وجود'

const userId = req.user.userId;
const isOwner = discussion.author._id.toString() === userId;
const currentUser = await User.findById(userId);
const isAdmin = currentUser && (currentUser.role === 'admin' || currentUser.r
if (!isOwner && !isAdmin) {
  return res.status(403).json({ success: false, message: 'ح بتعديل هذا النقاش'
}

const newTitle = req.body.title !== undefined ? sanitizeString(req.body.title
const newContent = req.body.content !== undefined ? sanitizeString(req.body.c
const reason = sanitizeString(req.body.reason || '', 200);

if (newTitle !== null && (!newTitle || newTitle.length < 5)) {
  return res.status(400).json({ success: false, message: 'العنوان قصير جداً'
}
if (newContent !== null && (!newContent || newContent.length < 10)) {
  return res.status(400).json({ success: false, message: 'المحتوى قصير جداً'
}

const titleChanged = newTitle !== null && newTitle !== discussion.title;
const contentChanged = newContent !== null && newContent !== discussion.conte

if (!titleChanged && !contentChanged) {
  return res.status(400).json({ success: false, message: 'لم يتغير شيء' });
}

discussion.editHistory.push({
  editedAt: new Date(),
  editedBy: { _id: currentUser._id, name: currentUser.name },
  previousTitle: discussion.title,
  previousContent: discussion.content,
  reason: reason,
});

if (titleChanged) discussion.title = newTitle;
if (contentChanged) discussion.content = newContent;
discussion.editedAt = new Date();
discussion.editsCount = (discussion.editsCount || 0) + 1;

await discussion.save();

```

```

    res.json({
      success: true,
      discussion: Object.assign({}, discussion.toObject(), {
        _id: discussion._id.toString(),
        isExpired: isDiscussionExpired(discussion)
      }),
      message: 'تم تعديل النقاش'
    });
  } catch (error) {
    console.error('Edit discussion error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.delete('/api/discussions/:id', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
    }
    const discussion = await Discussion.findById(req.params.id);
    if (!discussion) return res.status(404).json({ success: false, message: 'وجود'

    const userId = req.user.userId;
    const isOwner = discussion.author._id.toString() === userId;
    const currentUser = await User.findById(userId);
    const isAdmin = currentUser && (currentUser.role === 'admin' || currentUser.r
    if (!isOwner && !isAdmin) {
      return res.status(403).json({ success: false, message: 'موضح بحذف هذا النقاش'
    }

    await Comment.deleteMany({ discussionId: req.params.id });
    await Discussion.findByIdAndDelete(req.params.id);

    res.json({ success: true, message: 'تم حذف النقاش' });
  } catch (error) {
    console.error('Delete discussion error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.post('/api/discussions/:id/like', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).j
    const discussion = await Discussion.findById(req.params.id);
    if (!discussion) return res.status(404).json({ success: false, message: 'وجود'
    if (isDiscussionExpired(discussion)) return res.status(403).json({ success: f
    const userId = req.user.userId;

```



```

const likeIndex = discussion.upvotes.indexOf(userId);
const isLiking = likeIndex === -1;
if (isLiking) discussion.upvotes.push(userId);
else discussion.upvotes.splice(likeIndex, 1);
await discussion.save();

// 🛎 Send notification (only when liking, not unliking)
if (isLiking && discussion.author && discussion.author._id) {
  const user = await User.findById(userId);
  if (user) {
    await createNotification({
      recipient: discussion.author._id,
      sender: user,
      type: 'discussion_upvote',
      title: 'إعجاب جديد بنقاشك',
      message: `أعجب ${user.name} بنقاشك "${discussion.title}"`,
      link: `/discussion/${req.params.id}`,
      metadata: {
        discussionId: discussion._id,
      },
    });
  }
}

res.json({ success: true, liked: isLiking, upvotesCount: discussion.upvotes.length });
} catch (error) {
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.post('/api/discussions/:id/comments', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).json({ success: false, message: 'معرّف غير صالح' });
    const discussion = await Discussion.findById(req.params.id);
    if (!discussion) return res.status(404).json({ success: false, message: 'موجود' });
    if (isDiscussionExpired(discussion)) return res.status(403).json({ success: false, message: 'مقالات انتهت صلاحيتها' });

    const content = sanitizeString(req.body.content, 5000);
    const stance = req.body.stance;
    const parentCommentId = req.body.parentCommentId || null;

    if (!content) return res.status(400).json({ success: false, message: 'محتوى مطلوب' });

    let isReply = false;
    let validParentId = null;
    let finalStance = stance;

```

```

if (parentCommentId) {
  if (!mongoose.Types.ObjectId.isValid(parentCommentId)) {
    return res.status(400).json({ success: false, message: 'يق الأصلی غیر صحیح' })
  }
  const parentComment = await Comment.findById(parentCommentId);
  if (!parentComment) {
    return res.status(404).json({ success: false, message: 'ق الأصلی غیر موجود' })
  }
  if (parentComment.discussionId.toString() !== req.params.id) {
    return res.status(400).json({ success: false, message: 'ینتمی لنقاش مختلف' })
  }
  isReply = true;
  validParentId = parentComment._id;
  finalStance = parentComment.stance;
} else {
  if (!['pro', 'con', 'neutral'].includes(stance)) {
    return res.status(400).json({ success: false, message: 'وقف (مع/ضد/محايد)' })
  }
}

const user = await User.findById(req.user.userId);
if (!user) return res.status(404).json({ success: false, message: 'غير موجود' })

const newComment = await Comment.create({
  discussionId: req.params.id,
  content,
  stance: finalStance,
  author: {
    _id: user._id,
    name: user.name,
    avatar: user.avatar,
    reputation: user.reputation,
    isFoundingMember: user.isFoundingMember
  },
  reactions: {
    logical: [],
    illogical: [],
    inspiring: [],
    unclear: [],
  },
  qualityScore: 0,
  parentCommentId: validParentId,
  isReply,
});

if (!isReply) {
  const stanceField = `stanceStats.${finalStance}`;

```

```

    await Discussion.findByIdAndUpdate(req.params.id, {
      $inc: { commentCount: 1, [stanceField]: 1 }
    });
  } else {
    await Discussion.findByIdAndUpdate(req.params.id, {
      $inc: { commentCount: 1 }
    });
  }

  await updateAuthorReputation(user._id, REPUTATION_REWARDS.CREATE_COMMENT);

  // 🛎 Send notification
  if (isReply) {
    const parentComment = await Comment.findById(validParentId);
    if (parentComment && parentComment.author && parentComment.author._id) {
      await createNotification({
        recipient: parentComment.author._id,
        sender: user,
        type: 'reply',
        title: 'رَدّ جديد على تعليقك',
        message: `رَدّ ${user.name} على تعليقك في "${discussion.title}"`,
        link: `/discussion/${req.params.id}`,
        metadata: {
          discussionId: discussion._id,
          commentId: newComment._id,
        },
      });
    }
  } else {
    if (discussion.author && discussion.author._id) {
      await createNotification({
        recipient: discussion.author._id,
        sender: user,
        type: 'comment',
        title: 'تعليق جديد على نقاشك',
        message: `عَلّقَ ${user.name} على "${discussion.title}"`,
        link: `/discussion/${req.params.id}`,
        metadata: {
          discussionId: discussion._id,
          commentId: newComment._id,
        },
      });
    }
  }

  res.status(201).json({
    success: true,
  });
}

```

```

        comment: Object.assign({}, newComment.toObject(), {
            _id: newComment._id.toString(),
            parentCommentId: validParentId ? validParentId.toString() : null,
        })
    });
} catch (error) {
    console.error('Add comment error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.patch('/api/comments/:id', authMiddleware, async (req, res) => {
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
            return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
        }
        const comment = await Comment.findById(req.params.id);
        if (!comment) return res.status(404).json({ success: false, message: 'غير موجود' });

        const userId = req.user.userId;
        const isOwner = comment.author._id.toString() === userId;
        const currentUser = await User.findById(userId);
        const isAdmin = currentUser && (currentUser.role === 'admin' || currentUser.role === 'moderator');
        if (!isOwner && !isAdmin) {
            return res.status(403).json({ success: false, message: 'بتعديل هذا التعليق' });
        }

        const discussion = await Discussion.findById(comment.discussionId);
        if (discussion && isDiscussionExpired(discussion)) {
            return res.status(403).json({ success: false, message: 'انتهى وقت النقاش' });
        }

        const newContent = sanitizeString(req.body.content, 5000);
        if (!newContent || newContent.length < 1) {
            return res.status(400).json({ success: false, message: 'محتوى التعليق مطلوب' });
        }
        if (newContent === comment.content) {
            return res.status(400).json({ success: false, message: 'لم يتغير شيء' });
        }

        comment.content = newContent;
        comment.editedAt = new Date();
        await comment.save();

        res.json({
            success: true,
            comment: Object.assign({}, comment.toObject(), {

```

```

        _id: comment._id.toString(),
        parentCommentId: comment.parentCommentId ? comment.parentCommentId.toStri
    )),
    message: 'تم تعديل التعليق'
  });
} catch (error) {
  console.error('Edit comment error:', error);
  res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.post('/api/comments/:id/upvote', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
    }
    const comment = await Comment.findById(req.params.id);
    if (!comment) return res.status(404).json({ success: false, message: 'موجود' });

    const discussion = await Discussion.findById(comment.discussionId);
    if (discussion && isDiscussionExpired(discussion)) {
      return res.status(403).json({ success: false, message: 'انتهى وقت النقاش' });
    }

    const userId = req.user.userId;
    const upvoteIndex = comment.upvotes.indexOf(userId);
    const isUpvoting = upvoteIndex === -1;

    if (isUpvoting) {
      comment.upvotes.push(userId);
    } else {
      comment.upvotes.splice(upvoteIndex, 1);
    }

    comment.qualityScore = calculateQualityScore(comment);
    await comment.save();

    if (comment.author._id.toString() !== userId) {
      const reputationChange = isUpvoting ? REPUTATION_REWARDS.RECEIVE_UPVOTE : -
      await updateAuthorReputation(comment.author._id, reputationChange);

      // 🛎 Send notification (only when upvoting)
      if (isUpvoting) {
        const user = await User.findById(userId);
        if (user && discussion) {
          await createNotification({
            recipient: comment.author._id,

```

```

        sender: user,
        type: 'comment_upvote',
        title: 'إعجاب جديد بتعليقك',
        message: `أعجب ${user.name} بتعليقك في "${discussion.title}"`,
        link: `/discussion/${comment.discussionId}`,
        metadata: {
            discussionId: comment.discussionId,
            commentId: comment._id,
        },
    });
}
}
}

res.json({
    success: true,
    upvoted: isUpvoting,
    upvotesCount: comment.upvotes.length,
    qualityScore: comment.qualityScore
});
} catch (error) {
    console.error('Upvote comment error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

const REACTION_REPUTATION_MAP = {
    logical: 'RECEIVE_LOGICAL',
    illogical: 'RECEIVE_ILLOGICAL',
    inspiring: 'RECEIVE_INSPIRING',
    unclear: 'RECEIVE_UNCLEAR',
};

app.post('/api/comments/:id/react', authMiddleware, async (req, res) => {
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
            return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
        }

        const reactionType = req.body.type;
        const validReactions = ['logical', 'illogical', 'inspiring', 'unclear'];
        if (!validReactions.includes(reactionType)) {
            return res.status(400).json({ success: false, message: 'وع التفاعل غير صحيح' });
        }

        const comment = await Comment.findById(req.params.id);
        if (!comment) return res.status(404).json({ success: false, message: 'ر موجود' });
    }
});

```

```

const discussion = await Discussion.findById(comment.discussionId);
if (discussion && isDiscussionExpired(discussion)) {
  return res.status(403).json({ success: false, message: 'انتهى وقت النقاش' })
}

if (!comment.reactions) {
  comment.reactions = { logical: [], illogical: [], inspiring: [], unclear: []
}
validReactions.forEach(t => {
  if (!comment.reactions[t]) comment.reactions[t] = [];
});

const userId = req.user.userId;
const currentReactionIndex = comment.reactions[reactionType].indexOf(userId);
const isAdding = currentReactionIndex === -1;

let removedReactionType = null;

if (isAdding) {
  const userActiveReactions = validReactions.filter(t =>
    comment.reactions[t].includes(userId)
  );

  if (userActiveReactions.length >= MAX_REACTIONS_PER_USER) {
    const oldestType = userActiveReactions[0];
    const idx = comment.reactions[oldestType].indexOf(userId);
    if (idx !== -1) {
      comment.reactions[oldestType].splice(idx, 1);
      removedReactionType = oldestType;

      if (comment.author._id.toString() !== userId) {
        const oldKey = REACTION_REPUTATION_MAP[oldestType];
        await updateAuthorReputation(comment.author._id, -REPUTATION_REWARDS[
      ]
    )
  }
}

comment.reactions[reactionType].push(userId);

if (comment.author._id.toString() !== userId) {
  const newKey = REACTION_REPUTATION_MAP[reactionType];
  await updateAuthorReputation(comment.author._id, REPUTATION_REWARDS[newKe
}
} else {
  comment.reactions[reactionType].splice(currentReactionIndex, 1);

```

```

    if (comment.author._id.toString() !== userId) {
      const removedKey = REACTION_REPUTATION_MAP[reactionType];
      await updateAuthorReputation(comment.author._id, -REPUTATION_REWARDS[remo
    }
  }

comment.qualityScore = calculateQualityScore(comment);
comment.markModified('reactions');
await comment.save();

// 🛎 Send notification (only for positive reactions when added)
if (isAdding && comment.author._id.toString() !== userId) {
  const positiveReactions = ['logical', 'inspiring'];
  if (positiveReactions.includes(reactionType)) {
    const user = await User.findById(userId);
    if (user && discussion) {
      const reactionLabels = { logical: 'منطقياً', inspiring: 'ملهماً' };
      await createNotification({
        recipient: comment.author._id,
        sender: user,
        type: reactionType === 'logical' ? 'reaction_logical' : 'reaction_ins
        title: 'تفاعل إيجابي مع تعليقك',
        message: `وجد ${user.name} تعليقك ${reactionLabels[reactionType]} في
        link: `/discussion/${comment.discussionId}`,
        metadata: {
          discussionId: comment.discussionId,
          commentId: comment._id,
        },
      });
    }
  }
}

res.json({
  success: true,
  reactionType,
  active: isAdding,
  removedReactionType,
  counts: {
    logical: comment.reactions.logical.length,
    illogical: comment.reactions.illogical.length,
    inspiring: comment.reactions.inspiring.length,
    unclear: comment.reactions.unclear.length,
  },
  qualityScore: comment.qualityScore
});
} catch (error) {

```



```

    console.error('React to comment error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.delete('/api/comments/:id', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
    }
    const comment = await Comment.findById(req.params.id);
    if (!comment) {
      return res.status(404).json({ success: false, message: 'التعليق غير موجود' });
    }
    const userId = req.user.userId;
    const isOwner = comment.author._id.toString() === userId;
    const currentUser = await User.findById(userId);
    const isAdmin = currentUser && (currentUser.role === 'admin' || currentUser.r
    if (!isOwner && !isAdmin) {
      return res.status(403).json({ success: false, message: 'وح بحذف هذا التعليق' });
    }

    const discussionId = comment.discussionId;
    const stance = comment.stance;
    const isReply = comment.isReply;

    await Comment.deleteMany({ parentCommentId: comment._id });
    await Comment.findByIdAndDelete(req.params.id);

    const repliesDeletedCount = await Comment.countDocuments({ parentCommentId: c
    const totalDeleted = 1 + repliesDeletedCount;

    if (!isReply) {
      const stanceField = `stanceStats.${stance}`;
      await Discussion.findByIdAndUpdate(discussionId, {
        $inc: { commentCount: -totalDeleted, [stanceField]: -1 }
      });
    } else {
      await Discussion.findByIdAndUpdate(discussionId, {
        $inc: { commentCount: -totalDeleted }
      });
    }

    res.json({ success: true, message: 'تم حذف التعليق' });
  } catch (error) {
    console.error('Delete comment error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

```

```

    }
  });

app.get('/api/circles', async (req, res) => {
  try {
    const circles = await Circle.find().lean();
    res.json({ success: true, circles: circles.map(c => Object.assign({}, c, { _id
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.get('/api/circles/:id', async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).j
    const circle = await Circle.findById(req.params.id).lean();
    if (!circle) return res.status(404).json({ success: false, message: 'ر موجودة'
    res.json({ success: true, circle: Object.assign({}, circle, { _id: circle._id
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.post('/api/circles/:id/join', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).j
    const circle = await Circle.findById(req.params.id);
    if (!circle) return res.status(404).json({ success: false, message: 'ر موجودة'
    const userId = req.user.userId;
    const user = await User.findById(userId).select('name avatar').lean();
    const memberIndex = circle.memberIds.indexOf(userId);

    // If already a member, leave the circle
    if (memberIndex !== -1) {
      circle.memberIds.splice(memberIndex, 1);
      circle.members = Math.max(0, circle.members - 1);
      await circle.save();
      return res.json({ success: true, joined: false, status: 'left', members: ci
    }

    // For private circles, add to pendingRequests instead of memberIds
    if (circle.isPrivate) {
      const alreadyPending = circle.pendingRequests.some(r => r.userId === userId
      if (alreadyPending) {
        return res.json({ success: true, status: 'pending', message: 'يد المراجعة'
      }
      circle.pendingRequests.push({

```

```

        userId: userId,
        userName: user ? user.name : 'مستخدم',
        userAvatar: user ? user.avatar : '',
        requestedAt: new Date(),
        message: req.body.message || '',
    });
    await circle.save();

    // 🛎 Notify circle creator about pending request
    if (circle.createdBy && circle.createdBy._id) {
        await createNotification({
            recipient: circle.createdBy._id,
            sender: user ? { _id: userId, name: user.name, avatar: user.avatar } :
            type: 'circle_join_request',
            title: 'طلب انضمام جديد',
            message: `طلب ${user ? user.name : 'مستخدم'} للانضمام لدائرة "${circle.name}"`,
            link: `/circles/${circle._id}`,
            metadata: { circleId: circle._id },
        });
    }

    return res.json({ success: true, status: 'pending', message: 'ل طلب الانضمام' });
}

// For public circles, add directly
circle.memberIds.push(userId);
circle.members += 1;
await circle.save();
res.json({ success: true, joined: true, status: 'joined', members: circle.members });
} catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

// Approve a pending request (admin/owner only)
app.post('/api/circles/:id/approve/:userId', authMiddleware, async (req, res) =>
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).json({ success: false, message: 'id غير صحيح' });
        const circle = await Circle.findById(req.params.id);
        if (!circle) return res.status(404).json({ success: false, message: 'دائرة موجودة' });

        // Only admin or circle creator can approve
        const user = await User.findById(req.user.userId);
        const isAdmin = user && (user.role === 'admin' || user.role === 'moderator');
        const isCreator = circle.createdBy && circle.createdBy._id === req.user.userId;
        if (!isAdmin && !isCreator) {
            return res.status(403).json({ success: false, message: 'تملك صلاحية الموافقة' });
        }
    } catch (error) {
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

```

```

    }

    const pendingIndex = circle.pendingRequests.findIndex(r => r.userId === req.p
    if (pendingIndex === -1) {
        return res.status(404).json({ success: false, message: 'طلب غير موجود' });
    }

    // Move from pending to members
    circle.pendingRequests.splice(pendingIndex, 1);
    if (!circle.memberIds.includes(req.params.userId)) {
        circle.memberIds.push(req.params.userId);
        circle.members += 1;
    }
    await circle.save();

    // 🛎 Notify approved user
    await createNotification({
        recipient: req.params.userId,
        sender: user,
        type: 'circle_approved',
        title: 'تمت الموافقة على طلبك! 🎉',
        message: `${circle.name} "على انضمامك لدائرة ${user.name} وافق`,
        link: `/circles/${circle._id}`,
        metadata: { circleId: circle._id },
    });

    res.json({ success: true, message: 'تمت الموافقة' });
} catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

// Reject a pending request
app.post('/api/circles/:id/reject/:userId', authMiddleware, async (req, res) => {
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) return res.status(400).j
        const circle = await Circle.findById(req.params.id);
        if (!circle) return res.status(404).json({ success: false, message: 'ر موجودة'

        const user = await User.findById(req.user.userId);
        const isAdmin = user && (user.role === 'admin' || user.role === 'moderator');
        const isCreator = circle.createdBy && circle.createdBy._id === req.user.userI
        if (!isAdmin && !isCreator) {
            return res.status(403).json({ success: false, message: 'لا تملك صلاحية الرفض'
        }

        const pendingIndex = circle.pendingRequests.findIndex(r => r.userId === req.p

```

```

    if (pendingIndex === -1) {
      return res.status(404).json({ success: false, message: 'طلب غير موجود' });
    }

    circle.pendingRequests.splice(pendingIndex, 1);
    await circle.save();

    // 🛎 Notify rejected user (politely)
    await createNotification({
      recipient: req.params.userId,
      sender: user,
      type: 'circle_rejected',
      title: 'تحديث على طلب الانضمام',
      message: `لم يتم قبوله حالياً "${circle.name}" طلبك للانضمام إلى`,
      link: `/circles`,
      metadata: { circleId: circle._id },
    });

    res.json({ success: true, message: 'تم الرفض' });
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

// =====
// 🛎 NOTIFICATIONS ENDPOINTS
// =====
app.get('/api/notifications', authMiddleware, async (req, res) => {
  try {
    const limit = Math.min(parseInt(req.query.limit) || 20, 50);
    const page = parseInt(req.query.page) || 1;
    const filter = req.query.filter;

    const query = { recipient: req.user.userId };
    if (filter === 'unread') query.isRead = false;

    const total = await Notification.countDocuments(query);
    const notifications = await Notification.find(query)
      .sort({ createdAt: -1 })
      .limit(limit)
      .skip((page - 1) * limit)
      .lean();

    res.json({
      success: true,
      notifications: notifications.map(n => ({ ...n, _id: n._id.toString() })),
      pagination: {

```

```

        page,
        pages: Math.ceil(total / limit),
        total,
    },
    });
} catch (error) {
    console.error('Get notifications error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.get('/api/notifications/unread-count', authMiddleware, async (req, res) => {
    try {
        const count = await Notification.countDocuments({
            recipient: req.user.userId,
            isRead: false,
        });
        res.json({ success: true, count });
    } catch (error) {
        console.error('Unread count error:', error);
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

app.patch('/api/notifications/:id/read', authMiddleware, async (req, res) => {
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
            return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
        }
        const notification = await Notification.findOneAndUpdate(
            { _id: req.params.id, recipient: req.user.userId },
            { isRead: true },
            { new: true }
        );
        if (!notification) {
            return res.status(404).json({ success: false, message: 'الإشعار غير موجود' });
        }
        res.json({ success: true, notification });
    } catch (error) {
        console.error('Mark as read error:', error);
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

app.patch('/api/notifications/read-all', authMiddleware, async (req, res) => {
    try {
        const result = await Notification.updateMany(

```

```

        { recipient: req.user.userId, isRead: false },
        { isRead: true }
    );
    res.json({ success: true, modifiedCount: result.modifiedCount });
} catch (error) {
    console.error('Mark all as read error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
}
});

app.delete('/api/notifications/:id', authMiddleware, async (req, res) => {
    try {
        if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
            return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
        }
        const result = await Notification.findOneAndDelete({
            _id: req.params.id,
            recipient: req.user.userId,
        });
        if (!result) {
            return res.status(404).json({ success: false, message: 'الإشعار غير موجود' });
        }
        res.json({ success: true });
    } catch (error) {
        console.error('Delete notification error:', error);
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

app.get('/api/users/leaderboard', async (req, res) => {
    try {
        const users = await User.find().sort({ reputation: -1 }).limit(10).select('name');
        res.json({ success: true, users: users.map(u => Object.assign({}, u, { id: u._id }));
    } catch (error) {
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

app.get('/api/users/profile', authMiddleware, async (req, res) => {
    try {
        const user = await User.findById(req.user.userId);
        if (!user) return res.status(404).json({ success: false, message: 'غير موجود' });
        const userDiscussions = await Discussion.countDocuments({ 'author._id': user._id });
        res.json({ success: true, user: Object.assign({}, userToResponse(user), { discussions: userDiscussions });
    } catch (error) {
        res.status(500).json({ success: false, message: 'خطأ في الخادم' });
    }
});

```

```

});

app.put('/api/users/profile', authMiddleware, async (req, res) => {
  try {
    const updates = {};
    if (req.body.name !== undefined) updates.name = sanitizeString(req.body.name,
    if (req.body.bio !== undefined) updates.bio = sanitizeString(req.body.bio, 50
    if (req.body.avatar !== undefined) updates.avatar = sanitizeString(req.body.a
    const user = await User.findByIdAndUpdate(req.user.userId, updates, { new: tr
    if (!user) return res.status(404).json({ success: false, message: 'غير موجود'
    res.json({ success: true, user: userToResponse(user) });
  } catch (error) {
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.get('/api/users/:id', async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ success: false, message: 'معرف غير صحيح' });
    }
    const user = await User.findById(req.params.id).select('-password').lean();
    if (!user) return res.status(404).json({ success: false, message: 'غير موجود'

    const discussions = await Discussion.find({ 'author._id': user._id })
      .sort({ createdAt: -1 })
      .limit(20)
      .lean();

    const discussionCount = await Discussion.countDocuments({ 'author._id': user.

    res.json({
      success: true,
      user: {
        id: user._id.toString(),
        _id: user._id.toString(),
        name: user.name,
        avatar: user.avatar,
        bio: user.bio,
        reputation: user.reputation,
        role: user.role,
        isFoundingMember: user.isFoundingMember,
        discussionCount,
        createdAt: user.createdAt
      },
      discussions: discussions.map(d => Object.assign({}, d, {
        _id: d._id.toString(),

```



```

        isExpired: isDiscussionExpired(d)
      )))
    });
  } catch (error) {
    console.error('Get user by id error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم' });
  }
});

app.get('/api/search', async (req, res) => {
  try {
    const q = (req.query.q || '').toString().trim();

    if (!q || q.length < 2) {
      return res.json({
        success: true,
        discussions: [],
        users: [],
        message: 'اكتب حرفين على الأقل للبحث'
      });
    }

    const searchRegex = new RegExp(q.replace(/[\.*+?^${}()|[\]\|\|]/g, '\\$&'), 'i')

    const [discussions, users] = await Promise.all([
      Discussion.find({
        $or: [
          { title: searchRegex },
          { content: searchRegex },
          { tags: searchRegex }
        ]
      })
      .sort({ views: -1, createdAt: -1 })
      .limit(20)
      .lean(),

      User.find({
        $or: [
          { name: searchRegex },
          { bio: searchRegex }
        ]
      })
      .select('name avatar bio reputation role isFoundingMember')
      .sort({ reputation: -1 })
      .limit(10)
      .lean()
    ]);
  }
});

```

```

res.json({
  success: true,
  query: q,
  discussions: discussions.map(d => Object.assign({}, d, {
    _id: d._id.toString(),
    isExpired: isDiscussionExpired(d)
  })),
  users: users.map(u => Object.assign({}, u, { id: u._id.toString(), _id: u._
  }));
} catch (error) {
  console.error('Search error:', error);
  res.status(500).json({ success: false, message: 'خطأ في البحث' });
}
});

// Delete legacy/test circles (admin only)
app.get('/api/admin/cleanup-circles', async (req, res) => {
  try {
    const adminKey = req.query.key;
    if (adminKey !== 'aporia-cleanup-2026') {
      return res.status(403).json({ success: false, message: 'غير مصرح' });
    }

    // Delete circles with non-emoji icons (legacy data)
    const legacyIcons = ['Brain', 'Scale', 'Cpu', 'TrendingUp', 'Heart', 'BookOpe
    const result = await Circle.deleteMany({
      icon: { $in: legacyIcons }
    });

    res.json({
      success: true,
      deletedCount: result.deletedCount,
      message: `دائرة قديمة ${result.deletedCount} تم حذف`
    });
  } catch (error) {
    console.error('Cleanup error:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});

app.get('/api/admin/seed-circles', async (req, res) => {
  try {
    const adminKey = req.query.key;
    if (adminKey !== 'aporia-seed-2026') {
      return res.status(403).json({ success: false, message: 'غير مصرح' });
    }

    const CIRCLES_DATA = [

```

```

{
  name: 'الفلسفة الإغريقية',
  description: 'دراسة فلاسفة اليونان: سقراط، أفلاطون، أرسطو، والحكماء قبلهم',
  category: 'philosophy',
  icon: '🏛️',
  color: '#3b82f6',
  bannerColor: 'from-blue-500/20 to-blue-700/10',
  tags: ['سقراط', 'أفلاطون', 'أرسطو', 'فلسفة قديمة'],
  isPrivate: false,
},
{
  name: 'الفلسفة الإسلامية',
  description: 'تراث الكندي والفارابي وابن سينا وابن رشد والغزالي',
  category: 'philosophy',
  icon: '🌙',
  color: '#10b981',
  bannerColor: 'from-emerald-500/20 to-emerald-700/10',
  tags: ['ابن رشد', 'ابن سينا', 'الفارابي', 'الغزالي'],
  isPrivate: false,
},
{
  name: 'فلسفة العقل',
  description: 'الوعي، الإدراك، الذكاء الاصطناعي، وطبيعة الفكر',
  category: 'philosophy',
  icon: '🧠',
  color: '#8b5cf6',
  bannerColor: 'from-purple-500/20 to-purple-700/10',
  tags: ['وعي', 'ذكاء اصطناعي', 'إدراك', 'علم النفس'],
  isPrivate: false,
},
{
  name: 'الأخلاق المعاصرة',
  description: 'الأخلاق التطبيقية، البيوإيثيقا، أخلاقيات التكنولوجيا',
  category: 'ethics',
  icon: '⚖️',
  color: '#f59e0b',
  bannerColor: 'from-amber-500/20 to-amber-700/10',
  tags: ['أخلاق', 'بيوإيثيقا', 'تكنولوجيا', 'مجتمع'],
  isPrivate: false,
},
{
  name: 'فلسفة السياسة',
  description: 'العدالة، الحرية، الديمقراطية، والنظم السياسية',
  category: 'politics',
  icon: '🌐',
  color: '#ef4444',
  bannerColor: 'from-red-500/20 to-red-700/10',

```

```

tags: ['عدالة', 'حرية', 'ديمقراطية', 'دولة'],
isPrivate: false,
},
{
name: 'فلسفة العلم',
description: 'المنهج العلمي، الإستمولوجيا، تاريخ العلوم',
category: 'science',
icon: '🔬',
color: '#06b6d4',
bannerColor: 'from-cyan-500/20 to-cyan-700/10',
tags: ['علم', 'منهج', 'إستمولوجيا', 'معرفة'],
isPrivate: false,
},
{
name: 'فلسفة الفن والجمال',
description: 'الإستطيقا، النقد الفني، فلسفة الإبداع',
category: 'aesthetics',
icon: '🎨',
color: '#ec4899',
bannerColor: 'from-pink-500/20 to-pink-700/10',
tags: ['فن', 'جمال', 'إبداع', 'نقد'],
isPrivate: false,
},
{
name: 'الوجودية والحياة',
description: 'كيركغارد، نيتشه، سارتر، كامو، ومعنى الوجود',
category: 'existentialism',
icon: '🕊️',
color: '#64748b',
bannerColor: 'from-slate-500/20 to-slate-700/10',
tags: ['وجودية', 'نيتشه', 'سارتر', 'معنى'],
isPrivate: false,
},
},
];

const results = { created: [], existed: [] };

for (const circleData of CIRCLES_DATA) {
  const existing = await Circle.findOne({ name: circleData.name });
  if (existing) {
    results.existed.push(circleData.name);
  } else {
    const newCircle = new Circle({
      ...circleData,
      members: 0,
      memberIds: [],
      pendingRequests: [],
    });
  }
}

```

```

        discussionCount: 0,
        createdBy: { _id: 'system', name: 'AporiaLab' },
    });
    await newCircle.save();
    results.created.push(circleData.name);
  }
}

res.json({ success: true, results });
} catch (error) {
  console.error('Seed circles error:', error);
  res.status(500).json({ success: false, message: error.message });
}
});

app.get('/api/admin/reset-founders', async (req, res) => {
  try {
    const key = req.query.key || '';
    if (key !== 'aporialab2026') {
      return res.status(403).json({ success: false, message: 'غير مصرح' });
    }

    const foundingPhilosophers = [
      { name: 'Ibn Rushd', email: 'ibn.rushd@aporialab.space', bio: 'Andalusian p' },
      { name: 'Al-Kindi', email: 'alkindi@aporialab.space', bio: 'First of the Ar' },
      { name: 'Hypatia', email: 'hypatia@aporialab.space', bio: 'Hellenistic phil' },
      { name: 'Avicenna', email: 'avicenna@aporialab.space', bio: 'Ibn Sina. Fath' },
      { name: 'Socrates', email: 'socrates@aporialab.space', bio: 'The Athenian g' },
    ];

    let created = 0;
    let updated = 0;

    for (const p of foundingPhilosophers) {
      const hashedPassword = await bcrypt.hash('AporiaLab2026!Founder', 10);
      const avatarUrl = 'https://api.dicebear.com/7.x/avataaars/svg?seed=' + p.se

      const existing = await User.findOne({ email: p.email });
      if (existing) {
        await User.findByIdAndUpdate(existing._id, {
          name: p.name, bio: p.bio, reputation: p.reputation,
          role: p.role, avatar: avatarUrl, isFoundingMember: true
        });
        updated++;
      } else {
        await User.create({
          name: p.name, email: p.email, password: hashedPassword,

```

```

        authProvider: 'local', emailVerified: true,
        bio: p.bio, reputation: p.reputation, role: p.role,
        avatar: avatarUrl, isFoundingMember: true
    });
    created++;
}
}

res.json({ success: true, message: 'تم تحديث المفكرين المؤسسين', created, upd
} catch (error) {
    console.error('Reset founders error:', error);
    res.status(500).json({ success: false, message: 'خطأ في الخادم', error: error
}
});

app.get('/api/admin/cleanup-non-founders', async (req, res) => {
    try {
        const key = req.query.key || '';
        if (key !== 'aporialab2026') {
            return res.status(403).json({ success: false, message: 'غير مصرح' });
        }

        const nonFounders = await User.find({ isFoundingMember: { $ne: true } }).select('name email');
        const userIds = nonFounders.map(u => u._id);
        const userInfo = nonFounders.map(u => ({ name: u.name, email: u.email }));

        if (userIds.length === 0) {
            return res.json({ success: true, message: 'لا يوجد مستخدمين للحذف', deletedUsers: 0,
                deletedNames: [], deletedDiscussions: 0, deletedComments: 0 });
        }

        const deletedDiscussions = await Discussion.deleteMany({ 'author._id': { $in: userIds } });
        const deletedComments = await Comment.deleteMany({ 'author._id': { $in: userIds } });
        const deletedUsers = await User.deleteMany({ _id: { $in: userIds } });

        res.json({
            success: true, message: 'تم حذف كل المستخدمين غير المؤسسين',
            deletedUsers: deletedUsers.deletedCount || 0,
            deletedNames: userInfo,
            deletedDiscussions: deletedDiscussions.deletedCount || 0,
            deletedComments: deletedComments.deletedCount || 0
        });
    } catch (error) {
        console.error('Cleanup error:', error);
        res.status(500).json({ success: false, message: 'خطأ في الخادم', error: error
    }
});

```

```
app.use((req, res) => res.status(404).json({ success: false, message: 'المسار ' +
app.use((err, req, res, next) => {
  if (err.message === 'Not allowed by CORS') return res.status(403).json({ succes
  console.error(err.stack);
  res.status(500).json({ success: false, message: 'خطأ داخلي في الخادم' });
});

module.exports = app;
```